

GANisme — Veille scientifique sur les GAN

Réponses aux 15 questions de la phase de veille du projet **GANisme** (La Plateforme_ — M2).

1. À quelle problématique d'apprentissage répond initialement l'algorithme GAN ? Est-il efficace sur d'autres problématiques d'apprentissage ?

Problématique initiale

Le GAN a été introduit par **Ian Goodfellow et al. en 2014** (*Generative Adversarial Nets*, NeurIPS) pour répondre au problème de la **modélisation générative** : apprendre à échantillonner de nouvelles données réalistes issues d'une distribution inconnue **p_data**, à partir de données **non étiquetées** et en grande quantité. Il s'agit donc d'un problème d'**apprentissage non supervisé**.

La motivation précise de Goodfellow était de **contourner les calculs probabilistes intraitables** des méthodes par maximum de vraisemblance (estimation de la constante de normalisation, chaînes de Markov type MCMC des machines de Boltzmann). Le GAN remplace ces calculs par une simple **rétropropagation dans un jeu entre deux réseaux**.

Efficacité sur d'autres problématiques

Oui, le cadre s'est révélé très efficace sur des tâches connexes :

Problématique	Exemple de modèle
Traduction image → image (supervisée / non appariée)	Pix2Pix, CycleGAN
Super-résolution	SRGAN, ESRGAN
Inpainting (restauration de zones manquantes)	Context Encoders
Augmentation de données / données synthétiques	DCGAN, StyleGAN-ADA, CTGAN (données tabulaires)
Apprentissage semi-supervisé	SGAN, CatGAN (le discriminateur devient un classifieur à $K+1$ classes)
Détection d'anomalies	AnoGAN, GANomaly (l'erreur de reconstruction sert de score)
Adaptation de domaine	DANN, ADDA (l'adversarial sert à aligner deux domaines)

Non, il est nettement moins adapté à d'autres :

- **Données discrètes (texte)** : le gradient ne peut pas être rétropropagé à travers un échantillonnage de tokens discrets. Il faut des contournements (Gumbel-Softmax, REINFORCE / SeqGAN) qui restent instables. Les **Transformers autorégressifs** dominent largement ce champ.

- **Estimation de densité / calcul de vraisemblance** : un GAN ne donne pas accès à $p(x)$ (cf. question 5). Les flux normalisants ou les modèles autorégressifs sont préférables.
- **Tâches purement discriminatives** (classification, régression) : un CNN ou un Transformer classique est plus simple et plus performant.
- **Génération d'images de très haute qualité** : depuis ~2021, les **modèles de diffusion** (DALL-E 2/3, Stable Diffusion, Midjourney) ont supplanté les GAN sur la fidélité et la diversité, au prix d'un échantillonnage plus lent.

2. Expliquez le fonctionnement d'un Generative Adversarial Network

Un GAN est une **architecture d'apprentissage profond** qui entraîne **deux réseaux de neurones en compétition** afin de générer de nouvelles données ressemblant à celles d'un jeu d'entraînement donné (nouvelles images à partir d'une base d'images, musique originale à partir d'une base de chansons, etc.).

L'intuition : le faussaire et l'expert

- Le **générateur** joue le rôle d'un **faussaire** qui fabrique des faux billets.
- Le **discriminateur** joue le rôle d'un **expert** qui tente de repérer les faux.
- À chaque itération, le faussaire s'améliore pour tromper l'expert, et l'expert s'améliore pour le démasquer. À la fin, les faux sont indiscernables des vrais.

Formalisation : un jeu minimax à somme nulle

Les deux réseaux optimisent la même fonction de valeur, mais dans des directions opposées :

$$\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}} [\log D(x)] + E_{z \sim p_z} [\log(1 - D(G(z)))]$$

- **D maximise V** : il veut sortir 1 sur les vraies données et 0 sur les fausses.
- **G minimise V** : il veut que $D(G(z))$ soit proche de 1.

La boucle d'entraînement

À chaque itération, on alterne :

1. **k pas sur D** (souvent $k = 1$) : on lui montre un batch de vraies données (cible 1) et un batch de fausses (cible 0), et on met à jour ses poids par descente de gradient sur une entropie croisée binaire.
2. **1 pas sur G** : on génère un batch de fausses données, on les passe dans **D** (gelé), et on met à jour **G** pour maximiser l'erreur de **D**.

Deux résultats théoriques importants

- Pour un **G** fixé, le discriminateur optimal vaut : $D^*(x) = p_{\text{data}}(x) / (p_{\text{data}}(x) + p_g(x))$
- En injectant D^* dans **V**, on obtient : $C(G) = -\log 4 + 2 \cdot \text{JSD}(p_{\text{data}} || p_g)$ où **JSD** est la **divergence de Jensen-Shannon**. Le minimum global est atteint **si et seulement si** $p_g = p_{\text{data}}$, avec $C(G) = -\log 4 \approx -1,386$ et $D(x) = 0,5$ partout.

L'astuce « non-saturante »

En pratique, minimiser $\log(1 - D(G(z)))$ sature en début d'entraînement (D rejette facilement les faux, le gradient de G est quasi nul). On maximise donc $\log D(G(z))$ à la place : mêmes points fixes, mais des gradients beaucoup plus forts quand G est mauvais.

3. Quel est le rôle du générateur et du discriminateur ? Expliquez la différence entre les modèles discriminants et génératifs

Rôle des deux réseaux

Un GAN est dit **antagoniste** (*adversarial*) parce qu'il entraîne deux réseaux différents et les oppose l'un à l'autre.

Le générateur $G : z \rightarrow \hat{x}$

- Transforme un vecteur de bruit aléatoire en un échantillon synthétique.
- **Il ne voit jamais les vraies données.** Il n'apprend que par le signal renvoyé par le discriminateur.
- Objectif : faire converger sa distribution implicite p_g vers p_{data} .
- Architecture typique : convolutions transposées (*upsampling*), BatchNorm, ReLU, activation finale **tanh**.

Le discriminateur $D : x \rightarrow [0,1]$

- Classifieur binaire qui estime la probabilité qu'une entrée soit réelle.
- Il joue le rôle de **fonction de perte apprise** pour G : au lieu d'une distance fixe (MSE, L1) qui produirait des images floues, il fournit un critère qui s'adapte et se raffine au fil de l'entraînement.
- Il est **jeté à la fin** : seul G est conservé pour la production.
- Architecture typique : convolutions à stride, LeakyReLU, activation finale **sigmoid**.

Modèles discriminants vs génératifs

C'est une distinction fondamentale en apprentissage statistique, qui porte sur **ce que le modèle apprend à représenter** :

	Modèle discriminant	Modèle génératif
Ce qu'il modélise	La probabilité conditionnelle $P(y x)$	La distribution jointe $P(x, y)$ ou la distribution $P(x)$
Question posée	« À quelle classe appartient cette donnée ? »	« À quoi ressemblent les données de cette classe ? »
Ce qu'il apprend	Une frontière de décision entre classes	La structure interne de chaque classe
Peut générer ?	Non	Oui (par échantillonnage)
Exemples	Régression logistique, SVM, forêts aléatoires, CNN classifieur, BERT	Naïve Bayes, GMM, HMM, VAE, modèles autorégressifs (GPT), modèles de diffusion, GAN

Le couple canonique : régression logistique (discriminant) et Naïve Bayes (génératif) modélisent le même problème, mais le premier apprend directement $P(y|x)$ alors que le second apprend $P(x|y)$ et $P(y)$ puis applique le théorème de Bayes.

Le cas du GAN : il est remarquable parce qu'il **contient les deux**. Le discriminateur est un modèle discriminant classique, le générateur est un modèle génératif — et c'est la mise en compétition du premier qui permet d'entraîner le second.

4. Qu'est-ce qui est donné en entrée au générateur et au discriminateur d'un GAN ?

Entrée du générateur

Un **vecteur de bruit aléatoire** z (appelé *vecteur latent* ou *code latent*), tiré d'une distribution simple et fixe p_z :

- généralement une **gaussienne centrée réduite** $N(0, I)$, parfois une uniforme $U(-1, 1)$;
- de dimension typiquement **100** (DCGAN) à **512** (StyleGAN) ;
- c'est la **seule source d'aléa** du modèle : à chaque z différent correspond une sortie différente. C'est la « graine » de la création.

L'espace dans lequel vit z est l'**espace latent**. Une fois le GAN entraîné, cet espace devient structuré : des directions y correspondent à des attributs sémantiques (âge, sourire, orientation du visage...), ce qui permet l'interpolation et l'édition (cf. question 7).

Entrée du discriminateur

Le discriminateur reçoit **alternativement, ou dans le même batch** :

- des **données réelles** $x \sim p_{\text{data}}$ issues du jeu d'entraînement, étiquetées **1** ;
- des **données fausses** $\hat{x} = G(z)$ produites par le générateur, étiquetées **0**.

Les deux types d'entrées doivent avoir **exactement le même format** (dimensions, nombre de canaux, plage de valeurs — d'où l'usage de $\tanh$ en sortie de G avec des vraies images normalisées dans $[-1, 1]$).

Cas des variantes conditionnelles

Dans un **cGAN**, les deux réseaux reçoivent en plus une **information de conditionnement** y (étiquette de classe encodée en one-hot, plongement de texte, carte de segmentation), concaténée à leur entrée. C'est ce qui permet de contrôler *ce qui* est généré au lieu de subir un tirage totalement aléatoire.

5. Pourquoi les GAN sont-ils appelés modèles de densité implicite ?

Les GAN sont appelés **modèles de densité implicite** parce qu'ils sont capables de générer de nouvelles données **sans jamais calculer explicitement la fonction de densité de probabilité** qui régit ces données.

Le mécanisme

Le générateur définit une distribution p_g de manière **purement procédurale** : on tire $z \sim N(0, I)$, on applique la transformation déterministe G , et l'on obtient un échantillon $\hat{x} = G(z)$. La distribution p_g existe bien — c'est la *push-forward* de p_z par G — mais **elle n'est accessible que par échantillonnage**. On ne dispose d'aucune formule permettant d'évaluer $p_g(x)$ pour un x donné.

C'est le discriminateur qui fournit indirectement l'information sur l'écart entre p_g et p_{data} , sans jamais expliciter ni l'une ni l'autre.

Situation dans la taxonomie des modèles génératifs

D'après la taxonomie de Goodfellow (tutoriel NIPS 2016) :

Famille	Sous-famille	Exemples
Densité explicite	Tractable	Modèles autorégressifs (PixelRNN/CNN, GPT), flux normalisants (RealNVP, Glow)
	Approchée	VAE (borne variationnelle ELBO), machines de Boltzmann (MCMC)
Densité implicite	Échantillonnage direct	GAN , GSN

Conséquences pratiques

Avantages

- Aucune contrainte architecturale : G peut être n'importe quel réseau différentiable (pas d'inversibilité requise comme pour les flux, pas d'ordre séquentiel comme pour l'autorégressif).
- Pas de compromis lié à une borne variationnelle → des images **nettes** là où le VAE produit du flou.
- Génération en **une seule passe avant**, donc très rapide.

Inconvénients

- Impossible de calculer la log-vraisemblance d'une donnée → pas de score de densité directement exploitable pour la détection d'anomalies ou la compression.
- L'évaluation devient un problème en soi** : il faut recourir à des métriques indirectes (FID, IS — cf. question 7).
- Le mode collapse est indétectable par la perte elle-même.

6. Comment la rétropropagation a-t-elle lieu sur ce type de réseau de neurones ?

Dans un GAN, la rétropropagation ne s'effectue pas en un seul bloc, mais de manière **découplée et alternée** entre les deux réseaux. Le point clé est que **le générateur n'a jamais accès aux vraies images** : il utilise le discriminateur comme un « pont » pour recevoir ses gradients d'erreur.

À chaque itération, le processus se déroule en **deux phases successives**.

Phase 1 — Entraînement du discriminateur D

Les poids du générateur sont **gelés**.

1. Passe avant

- On présente à **D** des vraies images du dataset (cible = **1**).
- On présente à **D** des fausses images produites par **G** à partir de bruit (cible = **0**).
- **D** renvoie une probabilité pour chaque image.

2. Calcul de la perte — entropie croisée binaire sur les deux batchs : $L_D = -[\log D(x) + \log(1 - D(G(z)))]$

3. Rétropropagation — les gradients sont propagés **uniquement à travers D**.

4. Mise à jour — les poids de **D** sont ajustés pour mieux discriminer.

💡 **En pratique (PyTorch)** : on utilise `fake.detach()` pour détacher les fausses images du graphe de calcul de **G**. Sans cela, le gradient remonterait inutilement dans **G** lors de cette phase.

Phase 2 — Entraînement du générateur **G**

Les poids du discriminateur sont **gelés** : il ne sert plus que de **guide mathématique** pour transmettre l'erreur.

1. Passe avant — **G** génère un nouveau batch de fausses images, envoyées directement dans **D**.

2. Calcul de la perte — l'astuce est que la **cible idéale pour G est que D réponde 1** (c'est-à-dire que **D** prenne les fausses images pour des vraies) : $L_G = -\log D(G(z))$ (*formulation non-saturante*) Plus la note de **D** est proche de 0, plus la perte de **G** est élevée.

3. Rétropropagation « à travers » **D** — c'est l'étape caractéristique du GAN :

- l'erreur est calculée à la **sortie** de **D** ;
- le gradient traverse d'abord **tout le réseau de D** (sans modifier ses poids, puisqu'ils sont gelés), par application de la règle de dérivation en chaîne ;
- arrivé à l'**entrée** de **D** — qui correspond à la **sortie** de **G** — le gradient continue sa route et se rétropropage à l'intérieur de **G**.

Formellement, la chaîne s'écrit : $\partial L_G / \partial \theta_G = (\partial L_G / \partial D) \cdot (\partial D / \partial \hat{x}) \cdot (\partial \hat{x} / \partial \theta_G)$

4. Mise à jour — les poids de **G** sont ajustés. Grâce au gradient transmis par **D**, **G** sait exactement comment modifier ses paramètres pour que ses images paraissent plus réalistes au coup d'après.

Points d'attention

- Il faut **deux optimiseurs distincts** (`opt_D` et `opt_G`), chacun n'ayant connaissance que des paramètres de son réseau.
- Il faut remettre les gradients à zéro (`zero_grad()`) avant chaque phase, sinon ils s'accumulent entre les deux.
- Les deux réseaux **ne convergent pas vers un minimum** au sens classique, mais vers un **équilibre de Nash** — d'où l'instabilité caractéristique (cf. question 9).

7. De quelle manière les performances d'un GAN peuvent-elles être évaluées ?

L'évaluation d'un GAN est l'un des défis les plus complexes de l'apprentissage profond, car **il n'existe pas de fonction de perte objective unique** pour mesurer la qualité des données générées. En particulier, la valeur des pertes **L_D** et **L_G** **n'est pas corrélée à la qualité visuelle** : elles peuvent osciller sans rien indiquer d'utile.

Les performances s'évaluent selon deux critères complémentaires :

- la **fidélité** — les images ont-elles l'air réelles ?
- la **diversité** — le GAN couvre-t-il toute la variété du jeu de données, ou répète-t-il toujours la même chose ?

1. Métriques quantitatives

Métrique	Principe	Interprétation
FID (Fréchet Inception Distance)	Extrait les caractéristiques des images réelles et générées via Inception-v3, puis compare la distance statistique entre les deux distributions (gaussiennes multivariées)	Plus bas = meilleur. Métrique de référence. Capture fidélité <i>et</i> diversité. Sensible à la taille de l'échantillon ($\geq 10\,000$ images recommandées)
IS (Inception Score)	Vérifie que chaque image contient un objet clairement identifiable (fidélité) et que l'ensemble couvre un large éventail de catégories (diversité)	Plus haut = meilleur. Limite : ne compare jamais aux vraies images, et n'est pertinent que sur des domaines proches d'ImageNet
KID (Kernel Inception Distance)	Variante du FID sans hypothèse gaussienne	Plus bas = meilleur. Moins biaisé sur de petits échantillons
Precision / Recall pour GAN	Deux scores séparés pour diagnostiquer : la précision mesure le pourcentage d'images générées qui ressemblent à de vraies images (fidélité) ; le rappel mesure la capacité à couvrir toute la variété du jeu réel (diversité)	Permet de distinguer « images belles mais peu variées » de « images variées mais ratées » — ce que le FID seul confond

2. Évaluation humaine

Coûteuse et difficile à reproduire, elle reste néanmoins une **référence** pour juger du réalisme perçu.

- **Tests de perception visuelle** : on présente à des juges un mélange d'images réelles et générées et on leur demande d'identifier les fausses. Un GAN parfait obtiendrait un taux d'erreur proche de **50 %**.
- **Plateformes de crowdsourcing** : des services comme Amazon Mechanical Turk permettent de faire évaluer des milliers d'images par des panels d'utilisateurs pour obtenir une validation statistique du réalisme.

3. Évaluation qualitative et diagnostics

Ces approches vérifient le comportement interne du modèle et détectent les anomalies de convergence.

- **Interpolation dans l'espace latent** : on sélectionne deux points de l'espace latent et on génère les images le long du chemin continu entre eux. Si la transition est **fluide et progressive**, cela prouve que

le GAN a appris des concepts abstraits généralisables et n'a pas simplement mémorisé les images d'entraînement. Des sauts brutaux révèlent un espace latent mal structuré.

- **Détection du mode collapse** : phénomène où le générateur produit continuellement les mêmes images ou un nombre très restreint de variantes. On le repère visuellement, en calculant la similarité structurelle (SSIM) entre images générées, ou via un **rappel** effondré.
- **Recherche des plus proches voisins** : on prend une image générée et on cherche la plus similaire dans le jeu d'entraînement. Si elles sont identiques, le GAN fait du **surapprentissage** et plagie les données réelles au lieu d'en inventer de nouvelles.
- **Grille d'échantillons à z fixé** : on conserve un même lot de vecteurs latents tout au long de l'entraînement et on génère l'image correspondante à chaque époque. Cela donne un suivi visuel direct de la progression.

8. Quelle est la quantité de données nécessaire à l'entraînement d'un GAN ?

Il n'existe pas de seuil universel. En ordre de grandeur, un entraînement **from scratch** demande généralement de **plusieurs milliers à plusieurs dizaines de milliers d'images** pour obtenir des résultats de bonne qualité.

Repères par jeu de données de référence

Jeu de données	Taille	Nature
MNIST	60 000	Chiffres manuscrits 28×28, niveaux de gris — tâche simple
CIFAR-10	50 000	Images couleur 32×32, 10 classes
CelebA	~200 000	Visages 178×218
FFHQ (StyleGAN)	70 000	Visages haute résolution 1024×1024
ImageNet	~1,2 M	1 000 classes, très grande diversité

Facteurs qui font varier ce besoin

- **Complexité et résolution de la tâche** : générer des formes simples (MNIST) demande bien moins de données et de capacité que des visages photoréalistes en 1024×1024.
- **Diversité visuelle du domaine** : un domaine restreint (un seul type d'objet, un seul style pictural) peut se contenter de **1 000 à 5 000 images**. Plus le domaine est varié, plus il faut de données pour couvrir tous les modes.
- **Augmentation de données adaptée** : **StyleGAN2-ADA** (Karras et al., 2020) introduit une augmentation adaptative appliquée aux entrées du discriminateur, ce qui permet d'entraîner des GAN de qualité avec **quelques milliers d'images seulement** (résultats convaincants dès ~1 000 sur des domaines restreints), sans que les artefacts d'augmentation ne « fuient » dans les générations.
- **Transfer learning** : partir d'un modèle pré-entraîné (par exemple StyleGAN entraîné sur FFHQ) et le *fine-tuner* réduit drastiquement le besoin en données — quelques centaines d'images peuvent suffire pour une adaptation de style.

Le vrai risque n'est pas la quantité seule

Avec trop peu de données, le **discriminateur surapprend** : il mémorise le jeu d'entraînement, atteint une précision quasi parfaite, et cesse de fournir un gradient utile au générateur. Le GAN se met alors à **recopier** les images d'entraînement plutôt qu'à en inventer. C'est précisément ce problème que l'ADA vient corriger.

Dans le cadre du projet : le dataset de peintures étant de taille modérée, il sera pertinent de privilégier une résolution raisonnable (64×64 ou 128×128) et de recourir à l'augmentation de données, voire au dataset complémentaire de peintures abstraites proposé dans le sujet.

9. Commentez la convergence d'un GAN selon les performances de son générateur et de son discriminateur

La convergence d'un GAN dépend d'un **équilibre délicat** entre les deux réseaux. Contrairement à un réseau classique qui descend vers un minimum, un GAN cherche un **équilibre de Nash** dans un jeu à somme nulle — un point de selle, beaucoup plus difficile à atteindre et à maintenir.

L'état d'équilibre idéal

- **Performance équilibrée** : le discriminateur ne sait plus faire la différence entre données réelles et fausses, et renvoie $D(x) = 0,5$ partout.
- **Générateur optimal** : il produit des données dont la distribution p_g coïncide avec p_{data} .
- **Valeurs de perte correspondantes** : avec une entropie croisée binaire, $L_D \approx L_G \approx \log 2 \approx 0,693$.
- **Convergence** : l'apprentissage se stabilise — mais cet état théorique **reste rare en pratique**.

Les déséquilibres de performance

Cas 1 — Discriminateur trop fort

- Il rejette immédiatement toutes les fausses images avec une certitude totale ($D(G(z)) \approx 0$).
- Avec la perte saturante, le gradient du générateur **s'annule** (*vanishing gradient*) : G ne reçoit plus aucun signal d'apprentissage et se fige.
- *Remèdes* : perte non-saturante, réduction du taux d'apprentissage de D , lissage des étiquettes (*label smoothing* : cible 0,9 au lieu de 1), pertes alternatives (WGAN-GP, hinge loss).

Cas 2 — Générateur trop fort (ou discriminateur trop faible)

- G trouve une faiblesse de D et l'exploite en produisant toujours le même type d'échantillon.
- Cela provoque un **mode collapse** : effondrement de la diversité, le générateur ne couvre plus qu'une fraction de la distribution réelle.
- *Remèdes* : minibatch discrimination, entraînement de D sur plusieurs pas par pas de G , WGAN, *unrolled GAN*.

Cas 3 — Oscillation et non-convergence

- Les deux réseaux se poursuivent indéfiniment sans se stabiliser : G couvre un mode, D apprend à le détecter, G bascule sur un autre mode, et ainsi de suite en boucle.
- Les pertes oscillent sans tendance claire.

Un piège méthodologique

La valeur des pertes n'est pas un indicateur fiable de qualité. Une perte L_G qui baisse ne signifie pas que les images s'améliorent : elle peut simplement signifier que D s'est affaibli. C'est pour cette raison qu'il est indispensable de **suivre en parallèle une grille d'images générées à z fixé** et une **métrique externe comme le FID**, calculée toutes les N époques (cf. question 7).

10. Quels sont les avantages et les inconvénients d'un GAN ?

Avantages

- **Réalisme élevé** : les GAN produisent des images, des sons ou des signaux d'une grande netteté, très proches de la réalité. Là où un VAE, contraint par sa borne variationnelle et sa perte de reconstruction pixel à pixel, produit des sorties floues, le discriminateur pousse le générateur à produire des détails nets.
- **Apprentissage non supervisé** : aucune annotation manuelle n'est requise, le modèle apprend directement à partir de données brutes.
- **Création de données** : génération de nouveaux exemples originaux, utiles pour enrichir des bases d'entraînement, équilibrer des classes minoritaires, ou produire des données synthétiques préservant la confidentialité.
- **Échantillonnage très rapide** : une seule passe avant suffit à produire un échantillon, là où un modèle de diffusion demande des dizaines à des centaines d'étapes de débruitage.
- **Grande liberté architecturale** : G peut être n'importe quel réseau différentiable, sans contrainte d'inversibilité (flux normalisants) ni de génération séquentielle (autorégressifs).
- **Espace latent structuré et exploitable** : permet interpolation, édition sémantique et contrôle fin des attributs (notamment avec StyleGAN).

Inconvénients

- **Instabilité de l'entraînement** : l'équilibre entre générateur et discriminateur est difficile à maintenir — l'un peut facilement écraser l'autre (cf. question 9).
 - **Mode collapse** : le générateur peut se bloquer et produire toujours le même type de sortie au lieu de couvrir toute la diversité des données.
 - **Évaluation difficile** : absence de fonction de perte interprétable, recours obligé à des métriques externes coûteuses et imparfaites (FID, IS).
 - **Sensibilité extrême aux hyperparamètres** : taux d'apprentissage, choix de l'optimiseur (Adam avec $\beta_1 = 0,5$ est quasi obligatoire), architecture, taille de batch — un réglage légèrement différent peut faire diverger complètement l'entraînement.
 - **Coût en calcul** : l'entraînement demande beaucoup de puissance et de temps (GPU haut de gamme sur plusieurs jours pour les grands modèles).
 - **Pas de vraisemblance calculable** : impossible d'évaluer la probabilité d'une donnée sous le modèle (cf. question 5).
 - **Risques éthiques** : deepfakes, usurpation d'identité, désinformation (cf. question 14).
-

11. Le GAN a plusieurs variantes, donnez-en au moins 3 avec des exemples d'applications

Le GAN a donné naissance à de très nombreuses variantes spécialisées. En voici cinq parmi les plus marquantes.

1. Conditional GAN (cGAN) — Mirza & Osindero, 2014

Le cGAN introduit une **condition** (étiquette de classe, texte, carte de segmentation) en entrée des deux réseaux, afin de **contrôler le type de données généré** au lieu de subir un tirage totalement aléatoire.

Applications

- **Génération texte → image** : créer une image précise à partir d'une description textuelle (ex. « un chat bleu sur un skateboard »).
- **Colorisation automatique** : transformer des photos ou films historiques en noir et blanc en versions colorisées réalistes.
- **Génération par classe** : produire un chiffre précis sur MNIST, ou une catégorie précise d'objet.

2. Deep Convolutional GAN (DCGAN) — Radford et al., 2015

Le DCGAN intègre des **réseaux de neurones convolutifs** au sein de l'architecture du GAN. Cette structure rend le modèle nettement **plus stable** et particulièrement performant sur les données visuelles. C'est la base de référence pour tout projet de génération d'images.

Applications

- **Génération de visages réalistes** : portraits de personnes fictives pour jeux vidéo ou banques d'images.
- **Design de produit** : génération de concepts (chaussures, sacs, mobilier) pour inspirer les designers.
- **Génération d'œuvres picturales** — c'est l'architecture naturellement adaptée au présent projet.

3. CycleGAN — Zhu et al., 2017

Cette variante réalise une **traduction d'image à image sans paires d'entraînement**. Le modèle apprend à transférer les caractéristiques d'un domaine A vers un domaine B **sans disposer de la photo exacte correspondante dans les deux styles**, grâce à une *perte de cohérence cyclique* ($A \rightarrow B \rightarrow A$ doit redonner l'image de départ).

Applications

- **Transfert de style artistique** : transformer une photo de paysage en tableau « à la manière de » Van Gogh ou Monet.
- **Conversion de saison ou de météo** : simuler un paysage enneigé à partir d'une vidéo filmée en été — très utile pour entraîner les algorithmes de voitures autonomes sur des conditions rares.
- **Imagerie médicale** : conversion entre modalités (IRM ↔ scanner) sans acquisitions appariées.

4. StyleGAN / StyleGAN2 / StyleGAN3 — Karras et al. (NVIDIA), 2018-2021

Le StyleGAN introduit un **réseau de mapping** transformant z en un espace latent intermédiaire w désenchevêtré, et injecte le style à chaque niveau de résolution via l'**AdaIN**. Il en résulte un **contrôle ultra-précis des styles visuels à différentes échelles** : forme du visage et pose aux basses résolutions, expression et coiffure aux moyennes, texture de peau et couleur aux hautes.

Applications

- **This Person Does Not Exist** : génération de visages photoréalistes de personnes inexistantes.
- **Effets spéciaux et cinéma** : doublures numériques, rajeunissement ou vieillissement d'acteurs de manière photoréaliste.
- **Mode virtuelle** : génération de mannequins virtuels pour l'essayage de vêtements en ligne.

5. Wasserstein GAN (WGAN / WGAN-GP) — Arjovsky et al., 2017

Le WGAN remplace la divergence de Jensen-Shannon par la **distance de Wasserstein** (*earth mover's distance*). Le discriminateur devient un « critique » qui note sans sigmoïde. Avantage décisif : **la perte devient corrélée à la qualité des images** et fournit enfin un indicateur de convergence exploitable, tout en réduisant fortement le mode collapse. Le WGAN-GP y ajoute une pénalité de gradient plus stable que le *weight clipping* initial.

Applications : utilisé comme fonction de perte de remplacement dans quasiment tous les domaines ci-dessus, dès que l'entraînement d'un GAN classique s'avère instable.

12. Comparez un CNN et un GAN. Qu'est-ce qu'un DCGAN ? Quelle différence avec le vanilla GAN ?

1. Comparaison CNN vs GAN

Attention : **les deux notions ne sont pas de même nature**. Un CNN est une **architecture de réseau** ; un GAN est un **cadre d'apprentissage** (*framework*), un protocole d'entraînement qui met deux réseaux en compétition — et qui utilise très souvent des CNN comme briques de base. La comparaison porte donc sur leur usage typique.

Caractéristique	CNN (Convolutional Neural Network)	GAN (Generative Adversarial Network)
Nature	Architecture de réseau	Cadre / protocole d'entraînement
Type de modèle	Discriminatif (analyse et classifie le réel)	Génératif (crée de nouvelles données réalistes)
Objectif principal	Extraire des caractéristiques pour prédire une étiquette (reconnaître un chat)	Créer des données de toutes pièces à partir de bruit aléatoire (générer un faux visage)
Structure	Un seul réseau (convolutions, pooling, couches denses)	Deux réseaux en compétition : générateur G et discriminateur D
Entrée → Sortie	Image → classe ou probabilité	Bruit aléatoire → image synthétique complète
Supervision	Supervisé (nécessite des étiquettes)	Non supervisé (les étiquettes réel/faux sont générées automatiquement)
Fonction de perte	Fixe et explicite (cross-entropy, MSE)	Apprise et dynamique (le discriminateur est la perte)

Caractéristique	CNN (Convolutional Neural Network)	GAN (Generative Adversarial Network)
Optimisation	Descente vers un minimum	Recherche d'un équilibre de Nash (point de selle)
Convergence	Stable et monotone	Instable, oscillante

Le lien entre les deux : dans un GAN d'images, le discriminateur **est** un CNN classique, et le générateur est un CNN « à l'envers ». Les deux notions sont complémentaires, pas concurrentes.

2. Qu'est-ce qu'un DCGAN ?

Le **DCGAN** (*Deep Convolutional GAN*, Radford, Metz & Chintala, 2015) est une évolution majeure du GAN qui intègre directement des **couches convolutives** dans les deux réseaux. Il a permis de stabiliser considérablement l'entraînement pour la génération d'images.

- Le **générateur** utilise des **convolutions transposées** (*transposed convolutions*) pour agrandir progressivement un vecteur de bruit jusqu'à une image complète : $z \text{ (100)} \rightarrow 4 \times 4 \times 512 \rightarrow 8 \times 8 \times 256 \rightarrow 16 \times 16 \times 128 \rightarrow 32 \times 32 \times 64 \rightarrow 64 \times 64 \times 3$.
- Le **discriminateur** est un CNN classique qui réduit l'image par convolutions à stride jusqu'à un scalaire indiquant vrai ou faux.

3. Quelle différence avec le vanilla GAN ?

Le **vanilla GAN** désigne la version d'origine de Goodfellow (2014). La différence tient à l'**architecture interne des réseaux** et aux **techniques de stabilisation**.

a) L'architecture des couches

	Vanilla GAN	DCGAN
Couches	Uniquement entièrement connectées (perceptron multicouche)	Convolutives spatiales
Traitement de l'image	Aplatie en un long vecteur → perte de l'information spatiale (la proximité des pixels n'est plus représentée)	Préserve la structure 2D de l'image
Nombre de paramètres	Explose avec la résolution	Maîtrisé grâce au partage de poids
Résolution atteignable	Très limitée (MNIST 28×28)	64×64 et au-delà

b) Les règles de stabilisation introduites par le DCGAN

Le vanilla GAN est extrêmement instable (mode collapse, gradients qui disparaissent). Le DCGAN formule un ensemble de recommandations empiriques devenues standard :

- Suppression des couches de pooling**, remplacées par des convolutions à stride (G et D apprennent eux-mêmes leur propre sur/sous-échantillonnage).

- **Batch Normalization** dans les deux réseaux — sauf sur la couche de sortie de **G** et la couche d'entrée de **D**, où elle dégrade les résultats.
- **Suppression des couches entièrement connectées cachées** au profit d'architectures pleinement convolutives.
- **Activations spécifiques** : **ReLU** dans le générateur (sauf la sortie en **tanh**), **LeakyReLU** (pente 0,2) dans le discriminateur — pour éviter les gradients nuls sur les valeurs négatives.
- **Optimiseur Adam** avec $\text{lr} = 0,0002$ et $\beta_1 = 0,5$ au lieu de 0,9.

13. Quel est le modèle génératif d'un chatbot populaire ? À quelle famille appartient-il ? Par qui a-t-il été développé ?

Le modèle

Le chatbot en question est **ChatGPT**, qui repose sur les modèles **GPT** (*Generative Pre-trained Transformer*) — successivement GPT-3.5, GPT-4, GPT-4o et leurs évolutions.

La famille

Il appartient à la famille des **Transformers** (Vaswani et al., 2017, *Attention Is All You Need*), et plus précisément aux **modèles de langage autorégressifs de type *decoder-only***.

Le développeur

Il a été développé par **OpenAI**, entreprise américaine fondée en 2015.

Différence fondamentale avec un GAN

	GAN	GPT / Transformer
Principe	Compétition entre deux réseaux	Prédiction du token suivant
Mécanisme clé	Jeu minimax adversarial	Auto-attention (<i>self-attention</i>) sur le contexte
Type de densité	Implicite (cf. question 5)	Explicite et tractable — la vraisemblance est factorisée par la règle de chaîne : $P(x) = \prod P(x_i)$
Entraînement	Alternance D / G , instable	Maximum de vraisemblance, stable
Génération	Une seule passe avant	Séquentielle, token par token
Données	Continues (images, son)	Discrètes (texte)

Là où un GAN fonctionne par compétition, un Transformer prédit simplement le mot suivant en s'appuyant sur des mécanismes d'**auto-attention** qui pondèrent l'importance de chaque token du contexte. ChatGPT y ajoute une phase d'alignement par **RLHF** (*Reinforcement Learning from Human Feedback*), qui ajuste le modèle pré-entraîné sur des préférences humaines.

Élargissement

Les autres grandes familles de modèles génératifs profonds sont :

- les **VAE** (auto-encodeurs variationnels) ;
- les **flux normalisants** (RealNVP, Glow) ;
- les **modèles de diffusion** (DDPM, Stable Diffusion, DALL-E 2/3, Midjourney), aujourd'hui dominants pour la génération d'images ;
- les **modèles autorégressifs** (GPT, PixelCNN).

14. Donnez les dangers d'utilisation de modèles de Deep Learning tels que les GAN

L'utilisation des GAN et autres modèles génératifs comporte des risques majeurs, allant de la manipulation de l'information à la cybersécurité.

Menaces pour la société et l'information

- **Création de deepfakes** : génération de fausses vidéos ou photos ultra-réalistes de personnalités publiques ou d'anonymes, facilitant la désinformation, la manipulation politique et l'usurpation d'identité.
- **Fraude et arnaques** : faux profils sur les réseaux sociaux, faux documents (pièces d'identité, justificatifs, preuves d'achat), clonage vocal pour de l'ingénierie sociale ou de l'escroquerie financière (fraude au président).
- **Érosion de la confiance publique** : à force de ne plus pouvoir distinguer le vrai du faux, le public développe un scepticisme généralisé. Effet pervers redoutable — le *liar's dividend* : une preuve authentique peut désormais être discréditée en la qualifiant de deepfake.

Problèmes juridiques et éthiques

- **Atteinte à la vie privée et harcèlement** : utilisation de visages de personnes réelles sans leur consentement pour créer du contenu diffamatoire ou à caractère pornographique (*revenge porn*, contenus pédocriminels synthétiques).
- **Violation de la propriété intellectuelle** : les modèles sont entraînés sur d'immenses corpus contenant des œuvres protégées par le droit d'auteur, sans accord ni rémunération des créateurs originaux. Plusieurs procès sont en cours sur ce point.
- **Amplification des biais** : si les données d'entraînement contiennent des stéréotypes de genre, d'origine ou de classe, le modèle les reproduit et les **accentue**. Un GAN entraîné sur un corpus déséquilibré sous-représentera systématiquement les minorités.
- **Mémorisation des données d'entraînement** : un GAN sur-entraîné peut recracher quasi à l'identique des données d'entraînement, ce qui pose un problème critique sur des données personnelles ou médicales.

Risques techniques et de cybersécurité

- **Empoisonnement des données** (*data poisoning*) : des acteurs malveillants manipulent les données d'entraînement pour forcer le modèle à produire des résultats défectueux ou à introduire des portes dérobées.
- **Contournement des systèmes de sécurité** : génération d'images capables de tromper les algorithmes de reconnaissance faciale, les filtres anti-spam ou les systèmes de vérification d'identité (*presentation*

attacks).

- **Génération d'exemples adverses** : création automatisée d'entrées conçues pour faire échouer d'autres systèmes d'IA critiques (conduite autonome, diagnostic médical).

Impact environnemental

- **Empreinte carbone élevée** : l'entraînement des grands modèles génératifs nécessite une puissance de calcul colossale (GPU de pointe tournant pendant des semaines), donc une consommation énergétique et en eau massive.

Éléments de réponse

Face à ces risques, plusieurs pistes se développent : le **tatouage numérique** (*watermarking*) des contenus générés, les standards de provenance comme **C2PA**, les détecteurs automatiques de deepfakes (course permanente entre l'épée et le bouclier), et la régulation — notamment l'**AI Act** européen, qui impose des obligations de transparence sur les contenus générés par IA.

15. Question philosophique : quel est l'intérêt qu'une IA puisse générer des données ?

L'intérêt pratique immédiat

La donnée est le carburant de l'apprentissage automatique, et c'est aussi sa ressource la plus coûteuse : il faut la **collecter, la nettoyer, l'étiqueter et l'organiser** — un travail humain long et onéreux. Qu'une IA sache générer des données répond directement à cette contrainte, en permettant de :

- **produire des données synthétiques** pour entraîner d'autres modèles quand les données réelles manquent ;
- **enrichir la diversité** des jeux de données et rééquilibrer les classes minoritaires, ce qui réduit certains biais ;
- **simuler des scénarios rares ou dangereux** qu'on ne peut pas provoquer dans le réel — un accident de voiture autonome, une panne industrielle critique, une pathologie rare ;
- **protéger la vie privée**, en substituant des données artificielles à des données personnelles réelles (santé, finance) tout en préservant les propriétés statistiques utiles.

L'intérêt épistémologique

Mais l'enjeu dépasse la simple production. **Générer, c'est prouver qu'on a compris**. Un modèle discriminant peut réussir une classification en s'appuyant sur un détail superficiel ; un modèle génératif, lui, doit avoir intériorisé la **structure profonde** des données pour en produire de nouvelles instances cohérentes. Richard Feynman l'avait résumé : « *What I cannot create, I do not understand.* » La capacité générative est ainsi un **test de compréhension** bien plus exigeant que la capacité discriminante — et c'est sans doute pour cela que les modèles génératifs se sont imposés comme la voie principale vers des systèmes plus généraux.

L'intérêt créatif — et sa limite

Reste la question qui traverse ce projet : une IA qui génère des peintures **crée-t-elle** ? Un GAN ne produit rien *ex nihilo* : il interpole dans un espace latent appris à partir d'œuvres humaines existantes. Il recombine plus qu'il n'invente, et il n'a ni intention, ni contexte culturel, ni rien à dire. On peut y voir la limite qui sépare la

nouveauté statistique de la **création artistique** : le GAN produit du jamais-vu sans jamais viser à signifier quoi que ce soit.

Mais cette limite se discute. L'art humain aussi procède largement par assimilation et recombinaison d'influences — et l'intention peut être réintroduite du côté de celui qui choisit le corpus, règle le modèle, sélectionne et présente les sorties. Le générateur devient alors un **outil** dans une démarche artistique, au même titre que l'appareil photo l'a été en son temps, en déplaçant l'acte créatif de l'exécution vers le cadrage et le choix.

Le revers

Cette puissance a un coût. Une machine capable de produire du vrai-semblable à volonté fragilise le lien entre **image et preuve** qui structurait notre rapport au réel (cf. question 14). Et l'abondance de données synthétiques menace les modèles eux-mêmes : entraînés sur leurs propres productions, ils dégénèrent — c'est l'effondrement de modèle (*model collapse*). L'intérêt de générer des données est donc réel et considérable, mais il est indissociable de la question de savoir **ce que l'on garde comme référence au réel**.

Sources

Articles fondateurs

- Goodfellow et al., *Generative Adversarial Nets*, NeurIPS 2014
- Radford, Metz & Chintala, *Unsupervised Representation Learning with Deep Convolutional GANs* (DCGAN), 2015
- Mirza & Osindero, *Conditional Generative Adversarial Nets*, 2014
- Arjovsky, Chintala & Bottou, *Wasserstein GAN*, 2017
- Zhu et al., *Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks* (CycleGAN), 2017
- Karras et al., *A Style-Based Generator Architecture for GANs* (StyleGAN), 2018 ; *Training GANs with Limited Data* (ADA), 2020
- Heusel et al., *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium* (FID), 2017
- Vaswani et al., *Attention Is All You Need*, 2017
- Goodfellow, *NIPS 2016 Tutorial: Generative Adversarial Networks*

Base de connaissances fournie dans le sujet

- *A Gentle Introduction to Generative Adversarial Networks (GANs)* — Machine Learning Mastery
- *A Beginner's Guide to Generative Adversarial Networks (GANs)* — Pathmind
- *On the evaluation of Generative Adversarial Networks*
- *Tips for Training Stable Generative Adversarial Networks*
- *Developers Google, Machine Learning (cours avancés) : GAN*
- *Open Questions about Generative Adversarial Networks* — Distill.pub